

Question 12

Q: A digital image appears distorted due to improper image formation. Identify possible causes and suggest improvements based on image formation models.

Theoretical Concept: Image Formation Distortions

Distortions often arise from the camera lens geometry, leading to **Radial Distortion** (barrel or pincushion effect) and **Tangential Distortion** (lens is not parallel to the imaging plane).

Improvements: We use camera calibration models (like Zhang's method) to find the intrinsic camera matrix and distortion coefficients, which mathematically warp the image back to its true form.

OpenCV Python Solution

 Copy

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to visualize the output on screen

# Step 1: Create a perfect, straight-line grid (the real world)
grid = np.zeros((200, 200), dtype=np.uint8) # Create a blank black image array filled with zeros
grid[::20, :] = 255
grid[:, ::20] = 255
```

```
# Step 2: Define a camera matrix and extreme barrel distortion coefficients
K = np.array([[100, 0, 100], [0, 100, 100], [0, 0, 1]], dtype=float)
D = np.array([-0.005, 0, 0, 0], dtype=float) # Negative k1 causes barrel effect

# Step 3: Mathematically simulate the light bending through the distorted lens
h, w = grid.shape
mapx, mapy = cv2.initUndistortRectifyMap(K, D, None, K, (w,h), 5) # Generate undistortion maps from camera calibration
distorted = cv2.remap(grid, mapx, mapy, cv2.INTER_LINEAR) # Apply geometric remap to correct lens distortion

# Step 4: Display the lens physics
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(grid, cmap='gray'), plt.title('Real World Geometry') # Place this image in a subplot
plt.subplot(122), plt.imshow(distorted, cmap='gray'), plt.title('Lens Barrel Distortion') # Place this image in a subplot
plt.show() # Show the final plot window to the user
```

Expected Output

You will see a standard square grid next to a grid whose corners are mathematically pushed outward, mimicking the fisheye (barrel) distortion caused by cheap camera lenses.

How to Execute & Submit

1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.

- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.